

Билет 4. `std::condition_variable`. Семафоры. `std::counting_semaphore`. Реализация семафора через `condition_variable`

0. Формулировка билета

Условные переменные: `std::condition_variable`. **Семафоры, класс**
`std::counting_semaphore`. **Реализация семафора через** `condition_variable`.

Главная идея билета:

Мьютекс позволяет защитить общие данные, но сам по себе не даёт удобного способа “уснуть до наступления условия”.
`condition_variable` позволяет потоку эффективно ждать изменения некоторого состояния и просыпаться, когда другой поток это состояние изменил.

Ключевая схема:

```
thread #1:
    lock mutex
    while (!condition) {
        cv.wait(lock)
    }
    use shared state

thread #2:
    lock mutex
    change shared state
    unlock mutex
    cv.notify_one()
```

Самое важное правило:

```
cv.wait(lock, predicate);
```

эквивалентно:

```
while (!predicate()) {
    cv.wait(lock);
}
```

Именно `while`, не `if`.

1. Почему одного мьютекса недостаточно

В прошлом билете мы защищали общие данные через `std::mutex`.

Например:

```
std::mutex m;  
std::queue<int> q;
```

Если один поток кладёт данные в очередь, а другой забирает, можно защитить очередь мьютексом:

```
void push(int x) {  
    std::lock_guard<std::mutex> lock(m);  
    q.push(x);  
}  
  
bool try_pop(int& result) {  
    std::lock_guard<std::mutex> lock(m);  
  
    if (q.empty()) {  
        return false;  
    }  
  
    result = q.front();  
    q.pop();  
  
    return true;  
}
```

Но здесь `try_pop` просто возвращает `false`, если очередь пуста.

А часто хочется другое поведение:

```
если очередь пуста — поток должен уснуть  
когда другой поток добавит элемент — разбудить спящий поток
```

Мьютекс решает проблему взаимного исключения, но не решает красиво проблему ожидания события.

2. Модельная задача: счётный семафор

Семафор — это примитив синхронизации со счётчиком.

У него есть две главные операции:

```
acquire(); // занять один ресурс  
release(); // освободить один ресурс
```

Смысл:

```
count > 0:  
    acquire проходит и уменьшает count  
  
count == 0:  
    acquire ждёт  
  
release:  
    увеличивает count и будит ожидающий поток
```

Пример:

```
semaphore count = 3
```

значит, одновременно могут пройти 3 потока.

После трёх `acquire`:

```
count = 0
```

следующий поток будет ждать.

После `release`:

```
count = 1
```

один ожидающий поток может продолжить.

3. Чем семафор отличается от мьютекса

Мьютекс — это частный случай идеи “пускать ограниченное число потоков”, но с лимитом 1.

mutex:
одновременно в критическую секцию входит максимум 1 поток

counting semaphore:
одновременно может войти N потоков

Сравнение:

Примитив	Сколько потоков пропускает
<code>std::mutex</code>	1
<code>std::binary_semaphore</code>	1
<code>std::counting_semaphore<Max></code>	до <code>Max</code>
самодельный counting semaphore	сколько задано счётчиком

Пример применения семафора:

ограничить число одновременно обслуживаемых клиентов
ограничить число одновременных запросов к базе
ограничить число задач, использующих дорогой ресурс
ограничить число одновременных downloads/uploads

В сервере семафор можно использовать как защиту от наплыва клиентов: если разрешить максимум 10 клиентов, то перед обработкой клиента вызываем `acquire`, а после обработки — `release`.

4. Интерфейс семафора

Свой класс можно представить так:

```
class Semaphore {
public:
    explicit Semaphore(int initial);

    void acquire();
    void release();

private:
    int count_;
};
```

Инвариант:

count_ — количество свободных разрешений

Если `count_ > 0`, поток может пройти.

Если `count_ == 0`, поток должен ждать.

5. Наивная реализация через busy wait

Самая простая идея:

```
class Semaphore {
public:
    explicit Semaphore(int initial) : count_(initial) {}

    void acquire() {
        while (count_ == 0) {
            std::this_thread::yield();
        }

        --count_;
    }

    void release() {
        ++count_;
    }

private:
    int count_;
};
```

Почему это плохо?

Проблема 1. Data race

`count_` читается и изменяется из нескольких потоков без синхронизации.

```
while (count_ == 0) {}
--count_;
```

и

```
++count_;
```

могут выполняться одновременно.

Это data race, то есть undefined behavior.

Проблема 2. Проверка и изменение не атомарны

Даже если забыть про C++ memory model, логически есть проблема.

Пусть:

```
count_ = 1
```

Два потока одновременно вызывают `acquire`.

```
thread #1 видит count_ == 1
thread #2 видит count_ == 1
thread #1 делает --count_
thread #2 делает --count_
```

Итог:

```
count_ = -1
```

То есть в критическую секцию прошли два потока, хотя ресурс был один.

Проблема 3. Busy wait тратит CPU

```
while (count_ == 0) {
    std::this_thread::yield();
}
```

Поток не спит нормально. Он постоянно просыпается и проверяет условие.

Даже с `yield` это не лучший вариант:

```
CPU тратится впустую
поток постоянно планируется
на однопроцессорной системе это может мешать потоку, который должен вызвать
release
```

6. Попытка через mutex и sleep

Исправим data race через мьютекс:

```

class Semaphore {
public:
    explicit Semaphore(int initial) : count_(initial) {}

    void acquire() {
        std::unique_lock<std::mutex> lock(m_);

        while (count_ == 0) {
            lock.unlock();
            std::this_thread::sleep_for(std::chrono::milliseconds(10));
            lock.lock();
        }

        --count_;
    }

    void release() {
        std::lock_guard<std::mutex> lock(m_);
        ++count_;
    }

private:
    int count_;
    std::mutex m_;
};

```

Теперь data race нет: `count_` защищён мьютексом.

Но остаются проблемы.

Проблема 1. Угадываем время сна

Если спать слишком мало:

много лишних пробуждений
нагрузка на CPU

Если спать слишком долго:

ресурс уже освободился,
а поток проснётся только через 10ms
лишняя задержка

Проблема 2. Нет точного уведомления

Поток, вызывающий `release`, просто увеличивает `count_`.

Он не может сказать ожидающему потоку:

условие изменилось, проснись прямо сейчас

Нужен примитив:

уснуть до уведомления
разбудить ожидающий поток

Это и есть `condition_variable`.

7. Что такое `std::condition_variable`

`std::condition_variable` — условная переменная.

Она позволяет одному потоку ждать наступления условия, а другому потоку уведомить, что условие могло измениться.

Основные методы:

```
wait(lock);  
wait(lock, pred);  
wait_for(lock, duration);  
wait_until(lock, time_point);  
  
notify_one();  
notify_all();
```

Смысл:

```
wait          — заснуть и ждать уведомления  
notify_one    — разбудить один ожидающий поток  
notify_all    — разбудить всех ожидающих потоков
```

Важно:

`condition_variable` сама по себе не хранит условие.
Условие хранится в обычных переменных под мьютексом.

Например:


```
int count_;
std::mutex m_;
std::condition_variable cv_;
```

Условие:

```
count_ > 0
```

`cv_` только помогает ждать изменения этого условия.

8. Почему `condition_variable` всегда используется с мьютексом

Почти всегда структура такая:

```
std::mutex m;
std::condition_variable cv;
bool ready = false;
```

Ожидающий поток:

```
std::unique_lock<std::mutex> lock(m);

while (!ready) {
    cv.wait(lock);
}
```

Уведомляющий поток:

```
{
    std::lock_guard<std::mutex> lock(m);
    ready = true;
}

cv.notify_one();
```

Мьютекс нужен по трём причинам:

1. Безопасно проверить условие.

```
while (!ready)
```

`ready` — общая переменная. Её нужно читать под мьютексом.

1. Атомарно отпустить мьютекс и заснуть.

`cv.wait(lock)` делает специальную операцию:

```
атомарно:  
    unlock mutex  
    перейти в состояние ожидания
```

Если бы мы писали это руками:

```
lock.unlock();  
cv.wait_somewhat();
```

между `unlock` и фактическим засыпанием другой поток мог бы сделать `notify_one`, и уведомление потерялось бы.

1. После пробуждения снова захватить мьютекс.

Когда `wait` возвращает управление, поток снова владеет мьютексом. Поэтому он может безопасно проверять условие и работать с общими данными.

9. Что реально делает `cv.wait(lock)`

Важный внутренний смысл:

```
cv.wait(lock);
```

делает примерно:

1. Поток уже владеет `lock`.
2. `wait` атомарно отпускает `mutex` и переводит поток в сон.
3. Другой поток может захватить `mutex` и изменить состояние.
4. Другой поток вызывает `notify_one/notify_all`.
5. Ожидающий поток просыпается.
6. Перед возвратом из `wait` он снова захватывает `mutex`.
7. Только после этого `wait` возвращает управление.

Схема:

thread A	thread B
<code>lock(m)</code>	

```

while (!ready)
    wait(lock):
        unlock(m)
        sleep

                                lock(m)
                                ready = true
                                unlock(m)
                                notify_one()

        wake up
        lock(m)
continue

```

10. Неправильный вариант: `if` вместо `while`

Наивный код:

```

void acquire() {
    std::unique_lock<std::mutex> lock(m_);

    if (count_ == 0) {
        cv_.wait(lock);
    }

    --count_;
}

```

На первый взгляд кажется правильным:

если ресурса нет — ждём
после пробуждения уменьшаем count

Но это ошибка.

После пробуждения условие нужно проверить заново.

11. Почему `if` ломается: **spurious wakeup**

Стандарт разрешает `condition_variable::wait` вернуться без `notify`.

Это называется **spurious wakeup** — ложное пробуждение.

То есть поток может проснуться просто так.

Если код написан через `if`:

```
if (count_ == 0) {  
    cv_.wait(lock);  
}  
  
--count_;
```

то при ложном пробуждении мы сделаем:

```
--count_;
```

даже если `count_` всё ещё `0`.

И получим:

```
count_ = -1
```

Нарушен инвариант семафора.

12. Почему `if` ломается: другой поток успел забрать ресурс

Даже если пробуждение было настоящим, условие всё равно может стать ложным до того, как поток продолжит работу.

Сценарий:

```
count_ = 0  
  
thread A:  
    acquire()  
    if count_ == 0 → wait  
  
thread B:  
    release()  
    count_ = 1  
    notify_one()  
  
thread A:  
    проснулся, но ещё не получил mutex  
  
thread C:  
    acquire()  
    видит count_ = 1  
    --count_  
    count_ = 0
```

```
thread A:
    наконец получил mutex
    выходит из wait
    делает --count_
    count_ = -1
```

Поэтому `if` недостаточно.

13. Правильный вариант: `while`

Правильно:

```
void acquire() {
    std::unique_lock<std::mutex> lock(m_);

    while (count_ == 0) {
        cv_.wait(lock);
    }

    --count_;
}
```

Почему это работает?

После каждого пробуждения поток снова проверяет условие:

```
проснулся
захватил mutex
проверил count_ > 0
если нет — снова уснул
```

`while` защищает от:

```
spurious wakeup
другой поток успел забрать ресурс
notify_all разбудил слишком много потоков
```

14. `wait(lock, pred)`

В C++ есть удобная форма:

```
cv_.wait(lock, [&] {  
    return count_ > 0;  
});
```

Она эквивалентна:

```
while (!(count_ > 0)) {  
    cv_.wait(lock);  
}
```

То есть:

```
void acquire() {  
    std::unique_lock<std::mutex> lock(m_);  
  
    cv_.wait(lock, [&] {  
        return count_ > 0;  
    });  
  
    --count_;  
}
```

На экзамене можно прямо сказать:

`wait(lock, pred)` — это не магия. Это удобная запись цикла `while (!pred())`
`wait(lock);`.

15. Корректная реализация семафора через `condition_variable`

```
#include <condition_variable>  
#include <mutex>  
  
class Semaphore {  
public:  
    explicit Semaphore(int initial) : count_(initial) {}  
  
    void acquire() {  
        std::unique_lock<std::mutex> lock(m_);  
  
        cv_.wait(lock, [&] {  
            return count_ > 0;  
        });  
  
        --count_;
```

```

    }

    void release() {
        {
            std::lock_guard<std::mutex> lock(m_);
            ++count_;
        }

        cv_.notify_one();
    }

private:
    int count_;
    std::mutex m_;
    std::condition_variable cv_;
};

```

Инвариант:

count_ — количество доступных разрешений
count_ >= 0

acquire:

ждёт, пока count_ > 0
уменьшает count_

release:

увеличивает count_
будит одного ожидающего

16. Можно ли вызывать **notify_one** под мьютексом?

Вариант 1:

```

void release() {
    {
        std::lock_guard<std::mutex> lock(m_);
        ++count_;
    }
}

```

```
    cv_.notify_one();  
}
```

Вариант 2:

```
void release() {  
    std::lock_guard<std::mutex> lock(m_);  
    ++count_;  
    cv_.notify_one();  
}
```

Оба варианта могут быть корректными.

Но часто предпочитают первый:

```
сначала изменить состояние под lock  
потом отпустить lock  
потом notify
```

Почему?

Если вызвать `notify_one` под мьютексом, ожидающий поток может проснуться, но тут же заблокироваться на попытке снова захватить этот же мьютекс.

То есть иногда эффективнее:

```
unlock → notify
```

Но главное правило не в этом.

Главное:

Сначала нужно изменить состояние, из-за которого предикат станет истинным, и только потом уведомлять.

То есть плохо:

```
cv_.notify_one();  
++count_;
```

Поток может проснуться, увидеть старое состояние и снова уснуть.

17. `notify_one` или `notify_all`

`notify_one`

```
cv_.notify_one();
```

Будит один ожидающий поток.

Для семафора, где `release` увеличивает счётчик на 1, обычно достаточно `notify_one`.

Почему?

появилось одно разрешение
нужен один поток

Плюс:

меньше лишних пробуждений
меньше конкуренции за mutex

`notify_all`

```
cv_.notify_all();
```

Будит всех ожидающих.

Это нужно, когда изменение состояния может позволить продолжить работу многим потокам.

Примеры:

stop = true в thread pool
очередь закрывается
барьер открывается для всех
появилось много ресурсов сразу

Минус `notify_all`:

thundering herd

То есть проснулись все, но пройти смог один или несколько, остальные снова уснут. Это лишняя нагрузка.

18. `notify_one` в семафоре: важная тонкость

Реализация:

```
void release() {  
    {  
        std::lock_guard<std::mutex> lock(m_);  
        ++count_;  
    }  
  
    cv_.notify_one();  
}
```

работает корректно.

Даже если два `release` происходят подряд, каждый вызывает `notify_one`.

Например:

```
A и B спят в acquire  
count_ = 0  
  
release #1:  
    count_ = 1  
    notify_one()  
  
release #2:  
    count_ = 2  
    notify_one()
```

В итоге есть два разрешения и два уведомления.

Даже если один поток проснулся позже, предикат `count_ > 0` защищает корректность.

19. Почему `condition_variable` не “помнит” `notify`

Это очень важная идея.

`notify_one` не сохраняется как событие само по себе.

Если никто не ждёт, то:

```
cv.notify_one();
```

просто ничего не разбудит.

Это нормально, потому что состояние хранится не в `cv`, а в переменной-предикате.

Например:

```
{
    std::lock_guard lock(m);
    ready = true;
}
cv.notify_one();
```

Если уведомление произошло до того, как поток начал ждать, поток потом сделает:

```
cv.wait(lock, [&] {
    return ready;
});
```

и не уснёт, потому что `ready == true`.

Поэтому правильно ждать не “уведомления”, а “условия”.

Формула:

```
condition_variable сообщает: “возможно, условие изменилось”
predicate проверяет: “условие действительно выполнено”
```

20. Lost wakeup

Lost wakeup — потерянное пробуждение.

Оно возникает, если поток проверяет условие и засыпает неатомарно.

Плохая идея:

```
lock.lock();

if (!ready) {
    lock.unlock();

    // между unlock и wait другой поток может сделать:
    // ready = true; notify_one();

    cv.wait_without_lock(); // условный псевдокод
```

```
}  
  
lock.lock();
```

Сценарий:

```
thread A:  
    проверил ready == false  
    отпустил mutex  
  
thread B:  
    lock  
    ready = true  
    unlock  
    notify_one  
  
thread A:  
    только теперь начал ждать  
    уведомление уже потеряно  
    А спит вечно
```

`cv.wait(lock)` решает это, потому что атомарно:

```
отпускает mutex  
и переводит поток в ожидание
```

21. `condition_variable_any`

Обычный `std::condition_variable` работает с:

```
std::unique_lock<std::mutex>
```

То есть обычно:

```
std::mutex m;  
std::condition_variable cv;  
  
std::unique_lock<std::mutex> lock(m);  
cv.wait(lock);
```

Есть более общий вариант:

```
std::condition_variable_any
```

Он может работать с любым lockable-объектом, у которого есть `lock()` и `unlock()`.

Например, с пользовательским мьютексом или `std::shared_mutex` в некоторых сценариях.

Минус:

```
condition_variable_any обычно тяжелее обычной condition_variable
```

Для базового ответа достаточно сказать:

`condition_variable_any` — более общий, но менее специализированный вариант.

22. `std::counting_semaphore`

В C++20 появился готовый семафор:

```
#include <semaphore>

std::counting_semaphore<10> sem(10);
```

Общий вид:

```
std::counting_semaphore<MaxValue> sem(initial);
```

Где:

`MaxValue` — максимальное значение счётчика
`initial` — начальное число доступных разрешений

Методы:

```
sem.acquire();           // ждать и занять разрешение
sem.release();           // освободить разрешение
sem.try_acquire();       // попытаться без ожидания
sem.try_acquire_for(duration);
sem.try_acquire_until(time_point);
```

Пример:

```

#include <semaphore>
#include <thread>

std::counting_semaphore<10> sem(10);

void handle_client(int client_fd) {
    // обработка клиента
}

void serve_client(int client_fd) {
    sem.acquire();

    std::thread t([client_fd] {
        handle_client(client_fd);
        sem.release();
    });

    t.detach();
}

```

Но здесь есть проблема exception-safety: если `handle_client` бросит исключение или поток выйдет не там, где мы ожидали, `release` может не выполниться.

Лучше использовать RAII.

23. RAII-обёртка для семафора

```

#include <semaphore>

template <std::ptrdiff_t Max>
class SemaphoreGuard {
public:
    explicit SemaphoreGuard(std::counting_semaphore<Max>& sem)
        : sem_(sem)
    {
        sem_.acquire();
    }

    ~SemaphoreGuard() {
        sem_.release();
    }

    SemaphoreGuard(const SemaphoreGuard&) = delete;
    SemaphoreGuard& operator=(const SemaphoreGuard&) = delete;

private:

```

```
std::counting_semaphore<Max>& sem_;  
};
```

Использование:

```
std::counting_semaphore<10> sem(10);  
  
void worker(int client_fd) {  
    SemaphoreGuard guard(sem);  
  
    handle_client(client_fd);  
}
```

Теперь `release` вызовется автоматически при выходе из функции.

24. Осторожно: где именно делать `acquire`

Есть два разных смысла.

Вариант 1. Ограничить число одновременно обслуживаемых клиентов

```
while (true) {  
    int client_fd = accept(server_fd, nullptr, nullptr);  
  
    sem.acquire();  
  
    std::thread t([client_fd] {  
        handle_client(client_fd);  
        sem.release();  
    });  
  
    t.detach();  
}
```

Здесь главный поток не создаст новый worker, если все слоты заняты.

Минус:

главный поток может заблокироваться на `sem.acquire`
и временно не вызывать `accept`

Вариант 2. Принять клиента, но отказать при перегрузке

```
if (!sem.try_acquire()) {  
    write_all(client_fd, "server overloaded\n", 18);  
}
```

```
    close(client_fd);  
    continue;  
}
```

Потом:

```
std::thread t([client_fd] {  
    try {  
        handle_client(client_fd);  
    } catch (...) {  
        // log  
    }  
  
    sem.release();  
});
```

Так сервер может явно ответить клиенту, что он перегружен.

25. `binary_semaphore`

`std::binary_semaphore` — семафор со значениями 0 или 1.

```
std::binary_semaphore sem(1);
```

Он похож на мьютекс, но семантика другая.

У мьютекса есть идея владения:

```
кто lock сделал, тот должен unlock сделать
```

У семафора владения нет в том же смысле:

```
один поток может acquire  
другой поток может release
```

Это важное отличие.

Пример:

```
thread A ждёт события через acquire  
thread B сообщает о событии через release
```

26. Семафор как ограничитель клиентов

В лекционных пометках семафор описан как способ ограничить число потоков в критической секции и защититься от наплыва клиентов.

Пример:

```
#include <semaphore>
#include <thread>

std::counting_semaphore<100> slots(100);

void handle_client_limited(int client_fd) {
    slots.acquire();

    try {
        handle_client(client_fd);
    } catch (...) {
        slots.release();
        throw;
    }

    slots.release();
}
```

Лучше RAII:

```
class ClientSlot {
public:
    explicit ClientSlot(std::counting_semaphore<100>& slots)
        : slots_(slots)
    {
        slots_.acquire();
    }

    ~ClientSlot() {
        slots_.release();
    }

private:
    std::counting_semaphore<100>& slots_;
};

void handle_client_limited(int client_fd) {
    ClientSlot slot(slots);
    handle_client(client_fd);
}
```

Смысл:

одновременно не больше 100 клиентов внутри `handle_client`

27. Блокирующая очередь как пример применения `condition_variable`

Хотя `thread pool` — это отдельный билет 5, полезно увидеть, зачем `condition_variable` реально нужна.

Блокирующая очередь:

```
push:
    добавить элемент
    notify_one

pop:
    если очередь пустая — ждать
    когда появился элемент — забрать
```

Код:

```
#include <condition_variable>
#include <mutex>
#include <queue>

template <typename T>
class BlockingQueue {
public:
    void push(T value) {
        {
            std::lock_guard<std::mutex> lock(m_);
            q_.push(std::move(value));
        }

        cv_.notify_one();
    }

    T pop() {
        std::unique_lock<std::mutex> lock(m_);

        cv_.wait(lock, [&] {
            return !q_.empty();
        });
    }
};
```

```

        T value = std::move(q_.front());
        q_.pop();

        return value;
    }

private:
    std::queue<T> q_;
    std::mutex m_;
    std::condition_variable cv_;
};

```

Здесь `condition_variable` нужна, чтобы worker-поток не крутился в цикле:

```
while (q.empty()) {}
```

а нормально спал, пока задач нет.

28. Блокирующая очередь с `stop`

В реальном thread pool нужен способ остановить workers.

Если очередь пустая, worker может спать вечно.

Добавляем флаг:

```
bool stop_ = false;
```

Код:

```

template <typename T>
class BlockingQueue {
public:
    void push(T value) {
        {
            std::lock_guard<std::mutex> lock(m_);

            if (stop_) {
                throw std::runtime_error("queue stopped");
            }

            q_.push(std::move(value));
        }

        cv_.notify_one();
    }
};

```

```

bool pop(T& out) {
    std::unique_lock<std::mutex> lock(m_);

    cv_.wait(lock, [&] {
        return stop_ || !q_.empty();
    });

    if (q_.empty()) {
        return false;
    }

    out = std::move(q_.front());
    q_.pop();

    return true;
}

void stop() {
    {
        std::lock_guard<std::mutex> lock(m_);
        stop_ = true;
    }

    cv_.notify_all();
}

private:
    std::queue<T> q_;
    bool stop_ = false;

    std::mutex m_;
    std::condition_variable cv_;
};

```

Почему в `stop()` нужен `notify_all`?

Потому что остановиться должны все worker-потоки, которые могут спать в `pop`.

Если вызвать только `notify_one`, проснётся один, а остальные могут остаться спать навсегда.

29. Почему для `condition_variable` нужен `std::unique_lock`, а не `lock_guard`

`std::lock_guard` нельзя явно разблокировать.

А `condition_variable::wait` должна уметь:

временно отпустить mutex
уснуть
после пробуждения снова захватить mutex

Поэтому `wait` принимает:

```
std::unique_lock<std::mutex>&
```

Пример:

```
std::unique_lock<std::mutex> lock(m);  
cv.wait(lock);
```

`lock_guard` слишком простой: он освобождает мьютекс только в деструкторе.

30. Порядок изменения состояния и уведомления

Правильный шаблон:

```
{  
    std::lock_guard<std::mutex> lock(m);  
    shared_state = new_value;  
}  
  
cv.notify_one();
```

или:

```
{  
    std::lock_guard<std::mutex> lock(m);  
    shared_state = new_value;  
    cv.notify_one();  
}
```

Главное:

сначала изменить состояние
потом уведомить

Неправильно:

```
cv.notify_one();

{
    std::lock_guard<std::mutex> lock(m);
    shared_state = new_value;
}
```

Почему плохо?

Ожидающий поток может проснуться, проверить предикат, увидеть старое состояние и снова уснуть.

31. Таймауты: `wait_for`, `wait_until`

Иногда нельзя ждать бесконечно.

```
std::unique_lock<std::mutex> lock(m);

bool ok = cv.wait_for(lock, std::chrono::seconds(1), [&] {
    return ready;
});
```

Если `ok == true`:

предикат стал true

Если `ok == false`:

таймаут

`wait_until` ждёт до конкретного момента времени:

```
cv.wait_until(lock, deadline, pred);
```

Это полезно:

в сетевом сервере
в очередях
при graceful shutdown
при ожидании ответа с дедлайном

32. Типичные ошибки

Ошибка 1. Использовать `if` вместо `while`

Плохо:

```
if (q.empty()) {  
    cv.wait(lock);  
}
```

Правильно:

```
while (q.empty()) {  
    cv.wait(lock);  
}
```

или:

```
cv.wait(lock, [&] {  
    return !q.empty();  
});
```

Ошибка 2. Ждать “уведомления”, а не “условия”

Плохо думать:

```
notify_one был → значит, можно продолжать
```

Правильно:

```
notify_one был → возможно, условие изменилось  
надо проверить predicate
```

Ошибка 3. Изменять predicate без мьютекса

Плохо:

```
ready = true;  
cv.notify_one();
```

если `ready` читается ожидающим потоком под мьютексом.

Правильно:

```
{
    std::lock_guard lock(m);
    ready = true;
}

cv.notify_one();
```

Ошибка 4. Уведомить до изменения состояния

Плохо:

```
cv.notify_one();
ready = true;
```

Правильно:

```
ready = true;
cv.notify_one();
```

при этом `ready` должен быть защищён мьютексом.

Ошибка 5. Использовать `condition_variable` без мьютекса

`condition_variable` не предназначена для такого использования.

Она должна быть связана с состоянием, защищённым мьютексом.

Ошибка 6. В деструкторе thread pool сделать `stop = true` без lock

Если `stop` читается под мьютексом в worker-потоках, менять его тоже нужно под этим же мьютексом.

Плохо:

```
stop = true;
cv.notify_all();
```

Правильно:


```

{
    std::lock_guard lock(m);
    stop = true;
}

cv.notify_all();

```

33. Что сказать на экзамене: короткая версия

Можно отвечать так:

`std::condition_variable` — это примитив синхронизации, который позволяет потоку эффективно ждать наступления некоторого условия. В отличие от `busy wait` поток не крутится в цикле, а засыпает и просыпается по `notify_one` или `notify_all`.

Условная переменная всегда используется вместе с мьютексом и некоторым состоянием-предикатом. Например, в семафоре таким состоянием является `count > 0`. Поток берёт `std::unique_lock<std::mutex>`, проверяет условие и вызывает `cv.wait(lock)`. Внутри `wait` атомарно отпускает мьютекс и переводит поток в состояние ожидания, а при пробуждении снова захватывает мьютекс.

Очень важно использовать `while`, а не `if`, потому что возможны spurious wakeups, а также другой поток может изменить состояние между уведомлением и фактическим продолжением работы. Поэтому правильная форма — `cv.wait(lock, pred)`, что эквивалентно `while (!pred()) cv.wait(lock);`.

Семафор хранит счётчик доступных ресурсов. `acquire` ждёт, пока счётчик станет положительным, и уменьшает его. `release` увеличивает счётчик и будит ожидающий поток. В C++20 есть готовый `std::counting_semaphore`, а до него семафор можно реализовать через `std::mutex` и `std::condition_variable`.

`notify_one` будит один поток и обычно подходит, если появился один ресурс. `notify_all` будит всех и нужен, например, при остановке thread pool, когда нужно разбудить всех worker-ов.

34. Мини-код для ответа на доске

Семафор через `condition_variable`

```

#include <condition_variable>
#include <mutex>

class Semaphore {

```

```

public:
    explicit Semaphore(int initial) : count_(initial) {}

    void acquire() {
        std::unique_lock<std::mutex> lock(m_);

        cv_.wait(lock, [&] {
            return count_ > 0;
        });

        --count_;
    }

    void release() {
        {
            std::lock_guard<std::mutex> lock(m_);
            ++count_;
        }

        cv_.notify_one();
    }

private:
    int count_;
    std::mutex m_;
    std::condition_variable cv_;
};

```

Что объяснить:

count_ — число свободных разрешений
 m_ защищает count_
 cv_ позволяет спать, пока count_ == 0
 wait(lock, pred) защищает от spurious wakeup
 release увеличивает count_ и будит одного ждущего

Блокирующая очередь

```

template <typename T>
class BlockingQueue {
public:
    void push(T value) {
        {
            std::lock_guard<std::mutex> lock(m_);
            q_.push(std::move(value));
        }
    }
};

```

```

        cv_.notify_one();
    }

    T pop() {
        std::unique_lock<std::mutex> lock(m_);

        cv_.wait(lock, [&] {
            return !q_.empty();
        });

        T value = std::move(q_.front());
        q_.pop();

        return value;
    }

private:
    std::queue<T> q_;
    std::mutex m_;
    std::condition_variable cv_;
};

```

35. Мини-проверка

1. Зачем нужна `std::condition_variable`, если уже есть `std::mutex`?
2. Что хранит сама `condition_variable`: условие или механизм ожидания?
3. Почему `condition_variable` используется вместе с мьютексом?
4. Что делает `cv.wait(lock)`?
5. Почему `wait` принимает именно `std::unique_lock`, а не `std::lock_guard`?
6. Что происходит с мьютексом внутри `cv.wait(lock)`?
7. Что такое spurious wakeup?
8. Почему после пробуждения нужно перепроверять условие?
9. Почему в `wait` нужен `while`, а не `if`?
10. Что делает `cv.wait(lock, pred)`?
11. Что такое lost wakeup?
12. Почему нельзя сначала отпустить mutex, а потом отдельно вызвать "wait"?
13. Что делает `notify_one`?
14. Что делает `notify_all`?
15. Когда лучше использовать `notify_one`?
16. Когда нужен `notify_all`?
17. Почему `notify_one` не "запоминается", если никто не ждёт?
18. Что такое семафор?
19. Чем семафор отличается от мьютекса?
20. Что делает `acquire` у семафора?
21. Что делает `release` у семафора?
22. Почему наивный семафор через `while(count == 0) yield()` плох?
23. Почему вариант с `sleep_for(10ms)` лучше, но всё ещё плох?
24. Как реализовать семафор через `mutex` и `condition_variable`?

25. Почему в `acquire` нужно проверять `count_ > 0` под мьютексом?
26. Почему в `release` нужно менять `count_` под мьютексом?
27. Почему для семафора обычно достаточно `notify_one`?
28. Что такое `std::counting_semaphore`?
29. Что такое `std::binary_semaphore`?
30. Чем семафор отличается от `mutex` с точки зрения владения?
31. Как семафор можно использовать для ограничения числа клиентов?
32. Зачем в блокирующей очереди нужен `stop`?
33. Почему при `stop = true` обычно нужен `notify_all`?
34. Что такое `condition_variable_any`?
35. Какие типичные ошибки бывают при использовании `condition_variable`?

36. Самый короткий ответ, если времени совсем мало

`std::condition_variable` нужен, чтобы поток мог эффективно ждать наступления условия, а не крутиться в `busy wait`. Она используется вместе с мьютексом и некоторым состоянием-предикатом. `wait(lock)` атомарно отпускает мьютекс и усыпляет поток, а при пробуждении снова захватывает мьютекс.

Ждать нужно не уведомления, а условия. Поэтому правильный паттерн:

```
std::unique_lock<std::mutex> lock(m);
cv.wait(lock, [&] { return condition; });
```

Это эквивалентно циклу:

```
while (!condition) {
    cv.wait(lock);
}
```

Цикл нужен из-за `spurious wakeups` и потому, что другой поток может изменить состояние после уведомления.

Семафор хранит счётчик ресурсов. `acquire` ждёт, пока `count > 0`, и уменьшает счётчик. `release` увеличивает счётчик и вызывает `notify_one`. В C++20 есть готовый `std::counting_semaphore`, а вручную его можно реализовать через `std::mutex` и `std::condition_variable`.